Understanding Jev

A source-grounded introduction to fast probabilistic decisions in software

1 Executive orientation. Jev is a model from TypeSafe AI that evaluates supplied information against developer-defined questions and returns machine-readable decisions and probabilities. Its documented interface is for choosing, rating and judging propositions; it does not write an answer as a chatbot would. The company calls this a **System One model**. That is its product category, not an independently established scientific taxonomy. [EXTERNAL VERIFICATION: E1; PARAPHRASE: S2, S6]

Calling Jev a **decision model** describes this observable function. Calling it a classifier is appropriate for some tasks, but misses scoring and its reusable question interface. Calling it a non-LLM or a wholly new foundation architecture would imply more knowledge of its internals than the material supports. TypeSafe describes a post-training path from pretrained language models, while detailed architecture remains unavailable in the reviewed sources. [EXTERNAL VERIFICATION: E7; MISSING CONTEXT: S2, S7]

Jev in one minute

What goes in	What comes back	Who acts
Relevant state plus a bounded question or rubric.	A choice, rubric score or yes/no probability; distributions accompany Choice and Score.	Application code applies its policy and executes an allowed action.

The attraction: the transcripts show responsive interfaces and promote large speed and cost reductions. These are plausible reasons to investigate a specialized decision interface, not evidence that every workflow improves. **The central limitation:** a valid choice can still be wrong. A probability does not authorize an action or guarantee its correctness. [PARAPHRASE: S1, S2, S5; INTERPRETATION]

2 The problem space

Many applications need a small judgment rather than a paragraph: which queue should receive a ticket, how urgent is it, or whether a proposed action should be reviewed. Repeating long text generation for such decisions can add delay and expense. Fixed output structures simplify integration; uncertainty can help the application reserve ambiguous cases for review. At scale, even small per-record costs accumulate. [PARAPHRASE: S2, S6, S7]

The prerequisite is a well-defined decision. Classification assigns a category; scoring locates an item on a rubric; ranking orders items; routing selects a destination; action selection chooses the next permitted operation. These processes can share model outputs but have different success criteria. A high relevance score may support ranking, while a route requires an application rule that maps the result to a service. [INTERPRETATION]

Reading convention. QUOTE means exact source wording; PARAPHRASE restates it; INTERPRETATION adds analysis; EXTERNAL VERIFICATION checks an external source, including vendor documentation, and is not necessarily independent validation. MISSING CONTEXT marks an unresolved gap. TEACHING CHOICE marks this document’s explanatory design. Source IDs S1-S9 identify supplied segments; E1-E30 are linked references on page 16. Worked guidance and the extension assessment appear on pages 4-5 and 10.

Extended edition. Both added transcripts are now represented. The material expands application examples and practical understanding; it does not establish universal speed, correctness or safety. New S9 details distinguish model estimates from application rules. [INTERPRETATION: S8/S9]

Core vocabulary

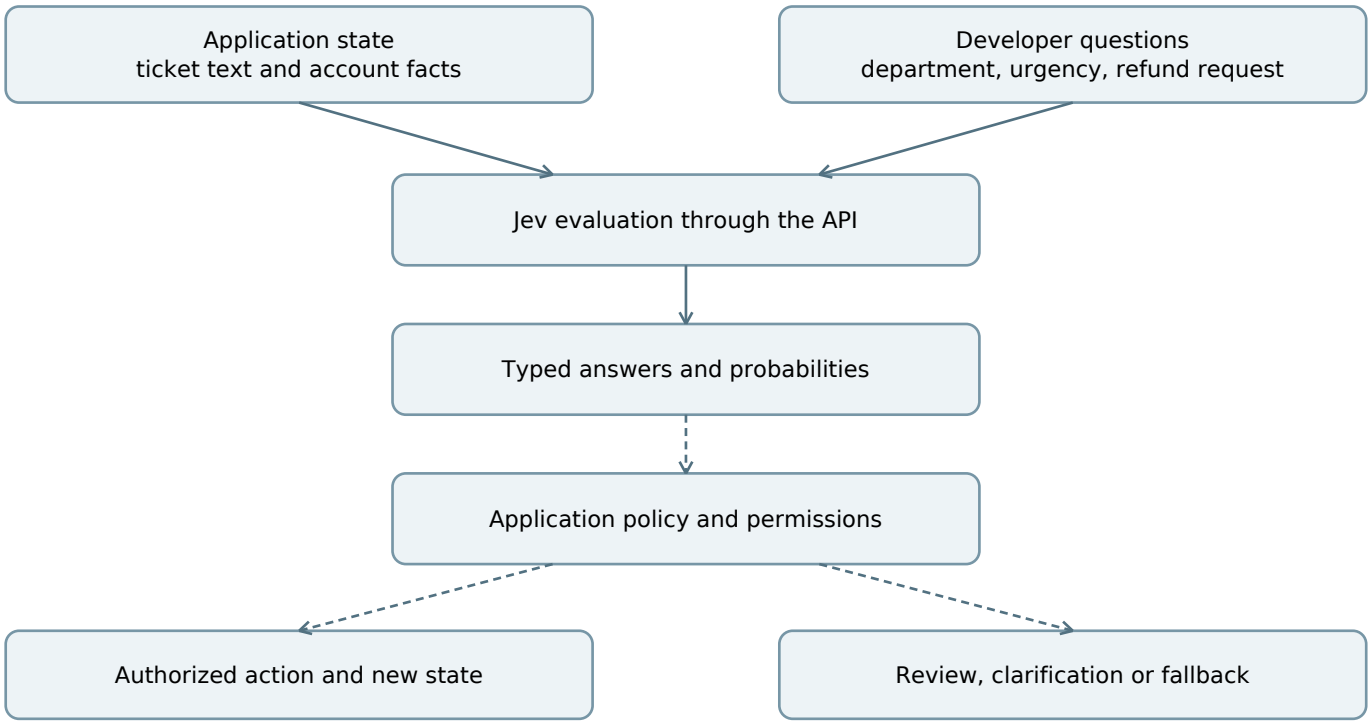
3 Concepts before implementation. Terms normalize transcript variants while retaining distinctions. Definitions are paraphrases; cues are TEACHING CHOICES. “Documented” means a public interface, not validated performance. IDs support appendix reuse.

ID and canonical term	Definition and recognition cue	Aliases and prerequisite	Source status
K1 Typed probabilistic decision	A bounded answer accompanied by uncertainty. **Cue:** values your code can use.	Typed decision; needs defined outcomes.	S2, S6; E1 documented
K2 Choice	Selects a named option and returns its distribution. **Cue:** which alternative?	choice; needs an option set.	S2; E2 documented
K3 Score	Probability-weighted position on ordered rubric levels. **Cue:** how much by this rubric?	score; needs ordered descriptions.	S2; E3 documented
K4 Noul	Probability that a yes/no proposition is true. **Cue:** how likely is yes?	“null” in S2/S4; Boolean in adapters; needs a proposition.	E4 corrected; S6
K5 Probability distribution	Mass assigned across possible outcomes, totaling one. **Cue:** how belief is divided.	probabilities; needs outcomes.	E2-E4 documented
K6 Calibration	Agreement between estimated probabilities and observed frequencies. **Cue:** does 80% mean about 80 in 100?	calibrated decisions; needs labeled outcomes.	E14 established concept; Jev performance unproven here
K7 Confidence	Summary of distribution concentration, distinct from a selected option’s probability. **Cue:** how concentrated?	confidence; needs K5.	E5 documented for Choice/Score
K8 Structured output	Data constrained to a declared schema. **Cue:** valid shape.	typed output; needs a schema.	S7; E6 documented
K9 Autoregression	Generation conditioned on preceding tokens. **Cue:** next token after previous tokens.	“auto reggressive”; needs token sequence.	S2, S4 paraphrase
K10 Latency	Elapsed time between specified start and finish points. **Cue:** how long for what?	response time; needs a measurement boundary.	S1, S7; E11 context
K11 Decision boundary	Rule separating outcomes or actions in input or score space. **Cue:** where the decision changes.	boundary; needs classes or scores.	Interpretation; not a disclosed Jev surface
K12 Null result	Absent value, distinct from a binary probability. **Cue:** no value present.	null/None in data; needs a schema.	E6; not the Noul primitive
K13 RLCD	Reinforcement Learning for Calibrated Decisions, TypeSafe’s named training method. **Cue:** train for uncertainty-aware decisions.	RLCD; needs K6 and learning objectives.	S3, S7; E7 vendor description
K14 RLHF and K15 RLVR	Reinforcement Learning from Human Feedback; Reinforcement Learning with Verifiable Rewards. **Cue:** preference versus checkable reward.	“RHF” → RLHF; needs reward optimization.	S3; E15/E7; distinct methods

Common misconception. A Noul of 0.8 is an estimated probability of yes, not “80% urgency.” Use a rubric score for degree. Noul has no separate confidence field; S7’s generic Boolean-plus-confidence illustration must not be copied as the native response schema. [EXTERNAL VERIFICATION: E4, E5]

The observable decision loop

4 How Jev works. The diagram shows the documented input/output contract and a proposed application policy. Solid arrows are supported interface relationships; dashed arrows are INTERPRETATION or TEACHING CHOICE. A warning marker “!” elsewhere identifies disputed or missing evidence. The diagram makes no claim about hidden neural layers. [E1, E6]



Prepare the state. Supply the facts needed for the question: text, records or structured observations. The structured request envelope does not require the state itself to be a rigid table; S4’s suggestion that raw text is excluded is too strong. **Define questions.** Name the options or ordered rubric, describe their meaning and include an “other” path when coverage is incomplete. [E1, E2; qualification of S4]

Evaluate and read. TypeSafe documents isolated evaluation of questions against shared state. Choice selects the highest-probability option; Score averages level indices by their probabilities; Noul returns a binary probability. With three score levels indexed 0, 1 and 2, hypothetical probabilities 0.1, 0.3 and 0.6 yield 1.5. This is an illustrative calculation, not a measured Jev response. [E2-E4; TEACHING CHOICE]

Apply policy, then act. For a support ticket, the application might route a sufficiently clear department classification, flag refund intent and request review when uncertain. A refund request is not refund eligibility; neither is permission to transfer money. Code must supply account checks, permissions and execution. An explicit policy can be deterministic even when the model’s estimates vary between calls, as S2 reports. [INTERPRETATION; S2]

Parallel does not mean dependency-free. Questions that share known state can be evaluated together. If a later decision needs a newly fetched record or a previous action’s result, the application still needs another step. Parallel evaluation also does not establish statistically independent errors. [INTERPRETATION based on E1; qualification of S6]

Speculative fan-out. A smart-home request can ask category, target and action together; code uses the relevant answers. S9 clarifies that questions cannot read each other’s answers. This is question batching, not parallel physical execution. See page 7. [E17; S9; INTERPRETATION]

One message, three judgments

4.1 Worked example from S9. The newly available tutorial makes the interface concrete by asking three different questions about one customer message. The example is pedagogically useful because intent, the presence of frustration and its degree are different judgments. [PARAPHRASE: S9; E2-E5]

QUOTE - S9: “I've contacted you three times and I'm still waiting.” This is the supplied state, not evidence of an actual customer incident.

Question and primitive	Answer meaning	How to read it
Does the message express frustration? **Noul**	Probability that the proposition is true.	0.94 favors yes. A value near 0 favors no; near 0.5 leaves yes/no unresolved. It does not measure the amount of frustration.
What does the customer want? **Choice**	Select from update, refund, replacement or other; retain all option probabilities.	A selected update label is an interpretation of intent. It is not permission to change an order or issue money.
How strongly is frustration expressed? **Score**	Use levels 0: no frustration; 1: dissatisfaction without strong anger; 2: strong anger.	The probability-weighted level can be fractional. Read the distribution as well as the mean.

Reported demonstration, not a test set. S9 describes a Noul near 0.94, an update Choice with confidence near 0.99, and a Score of 1.02 with confidence near 0.97. The transcript drops decimal points in places; the primitive definitions establish the 0-1 range. These are source-reported outcomes, not results reproduced for this edition. [PARAPHRASE: S9; EXTERNAL VERIFICATION: E2-E5]

What 1.02 means. If the level probabilities are 0, 0.98 and 0.02, then $0 \times 0 + 1 \times 0.98 + 2 \times 0.02 = 1.02$. That is an average over rubric positions. It is not a precise psychological measurement of a customer moving toward anger. Different distributions can share the same mean. [INTERPRETATION; E3]

The application supplies the rule. A high estimated probability of frustration may trigger a review flag. It cannot establish that the complaint is justified, that a service-level agreement was breached or that compensation is due. Those questions need additional facts and separate policy. [INTERPRETATION]

Why “other” matters. A closed set otherwise forces every input into one of the named categories. An other option handles unanticipated intent; an uncertain distribution signals a different problem. Neither automatically means that information is missing. [INTERPRETATION; S9; E2]

Teaching cue: Noul asks whether; Choice asks which; Score asks how much on a defined rubric. Confidence summarizes a distribution; calibration describes measured behavior across cases. [TEACHING CHOICE]

From the SDK to application policy

4.2 A minimal integration. S9 adds a TypeScript walkthrough, including credentials, the SDK client, question helpers and reading the response. The official JavaScript SDK confirms `@typesafe-ai/sdk` and `client.systemOne()`. The Python method is spelled differently: `system_one()`. [S9; E23; E4]

```
import { TypeSafeClient, noul } from "@typesafe-ai/sdk";

const client = new TypeSafeClient();
const result = await client.systemOne({
  state: { message: "My parcel is late; please help." },
  questions: {
    needsReview: noul("Does message express frustration?")
  }
});
const pYes = result.answers.needsReview.noul;
```

This is an illustrative SDK-shaped snippet, not an executed request. It assumes TYPESAFE_API_KEY is loaded server-side. The official SDK requires Node.js 20+; the tutorial repository uses Node.js 22+. Its `tsx` runner and ES-module setup are project choices, not Jev model capabilities. [EXTERNAL VERIFICATION: E23, E27, E30]

A question name such as needsReview is a lookup key. Its instruction must actually state what to evaluate; do not rely on the key to communicate the task. The response also records the serving model and token usage. A nonzero output-token count is compatible with zero output-token billing. It does not show an autoregressive prose-generation loop. [E2, E4; INTERPRETATION: S9]

Application branch	Illustrative policy	Meaning and boundary
High probability of yes	pYes ≥ 0.8: flag for review.	The tutorial uses 0.8 as an example, not an empirically validated cutoff.
Middle region	0.2 < pYes < 0.8: clarify or review.	Additional teaching branch. Ambiguity is handled explicitly.
Low probability of yes	pYes ≤ 0.2: leave in normal queue.	Likely no frustration, not a guarantee that the customer is satisfied.
No usable response	Handle failure; do not treat it as no.	Timeout, rate limit or malformed transport is a service event, not a model judgment.

INTERPRETATION / TEACHING CHOICE. These thresholds are deliberately illustrative. Use labeled messages to set the cost of missed complaints against unnecessary reviews. Test branching with fixed responses separately from testing whether Jev predicts correctly.

Batch only what shares evidence. S9 correctly explains that questions cannot read one another’s answers. If intent selection determines which order record to fetch, fetch it before the next evaluation. Question isolation is not proof of statistically independent errors. [S9; E1]

Reproducibility and limits. On 1 October, E24 lists jev-1.13.0, 64k total request tokens, and 32k for state plus the longest question. Pin a version when evaluating thresholds. Keep arithmetic, counts and date comparisons in code. [E24; E18]

Alternatives and trade-offs

5 Adjacent approaches. This comparison is a TEACHING CHOICE, not a benchmark ranking. Latency and calibration depend on the task and implementation. Discriminative models already classified without producing prose; BERT is established prior art. [E16]

Approach	Output and configuration	Speed and flexibility	Explanation, calibration and limits
Deterministic rules	Labels/actions from explicit conditions; no training.	Usually little computation; limited to encoded cases.	Inspectable logic; no learned uncertainty. Ambiguous language is difficult.
Conventional classifier	Classes or probabilities; usually labeled data and fitting.	Often fast for a fixed task; new labels can require retraining.	Interpretability varies; calibration and domain shift need testing.
Small specialized model	Labels, scores or text; task-specific adaptation.	Can reduce compute; size alone establishes no speed ordering.	Task evidence matters. Local control depends on availability and licensing.
Autoregressive chat model	Text/code or schema-constrained data; instructions and tools.	Flexible generation; output length and reasoning affect time.	Prose explanations are not proof. Schema validity does not ensure truth.
Jev-style decision model	Choice, Score, Noul; define state, questions and policy.	Designed for bounded judgments; no prose through this interface.	Distributions do not explain causes. Test accuracy and calibration locally. [S2, S7]

6 Capabilities and limits

Documented capabilities: mixed question types; up to 255 Choice options and 2-10 Score levels. Current TypeSafe limits are 64k total tokens and 32k for state plus the longest question; inputs are text only. These are interface limits, not parameter counts. [E2/E3/E24; 1 Oct 2026]

Claims versus examples: broad calibration, architectural novelty and large economic gains remain claims requiring scope. Forms, browser actions, games and the new smart-home example are demonstrations. Universal superiority, safe self-driving and zero harmful decisions remain unsupported. [S1-S5, S7, S8; INTERPRETATION]

Documented weaknesses, version-specific: TypeSafe lists nine failure modes for **jev-1.13**, including literal interpretation, numeric precision, date ordering, indirection, irrelevant context, adversarial content, conflicting criteria, inconsistent related answers and generation. Keep arithmetic and date comparisons in code. A Noul and a similarly worded Choice need not agree; separately asked propositions need not obey complementary-probability identities. [EXTERNAL VERIFICATION: E18, reviewed there 17 September 2026]

Unsuitable alone: writing explanations, arbitrary code, open-ended arguments or missing evidence. Closed-set function arguments can be selected when the application supplies candidates; this is not unrestricted argument generation. [S2, S7; E21]

System reliability: validate observations, permissions and action results; handle API errors separately from uncertain answers. S1 shows unexpected form judgments, S2 reports output variation, and S8 supplies no controlled security evaluation. Replacing a guardrail model does not itself establish safer automation. [S1, S2, S8; E6; INTERPRETATION]

Use cases and evidence boundaries

7 Application families. D = described demo; I = documented implementation; P = proposal. Suitability and prerequisites are INTERPRETATION. S8 extends U4/U9 and adds U12/U13; S9 deepens U2 and adds candidates on page 8.

ID / goal	Input → output	Prerequisite and possible fit	Limit / evidence
U1 Label records	Email, product or language text → Choice/Score.	Clear labels, representative cases; repeated bounded judgments.	Ambiguity/rare classes. D/P: S1, S2, S4, S8.
U2 Triage support	Message/account facts → queue, urgency, refund intent.	Queue definitions, rubric, escalation; shared context.	Intent is not eligibility. D/I: S1, S2, S6, S8; E12.
U3 Rank/enrich	Rows or retrieved candidates → scores/labels.	Consistent rubric, sorting code; repeated comparisons.	No missing facts supplied; scores need validation. D/P: S1, S4, S7.
U4 Route models, tools, skills	Request/catalog → model, tool or suggested skill.	Capability registry and fallback; inexpensive selection.	Selection is not execution. D/I/P: S2, S5, S6, S8; E12, E20, E21.
U5 Control workflow	Trace/proposed call → retry, stop, review, risk.	Explicit policy, permissions; checks at transitions.	Guardrail can miss harm. I/P: S2, S6-S8; E9, E12.
U6 Navigate UI	Observed controls/goal → operation and target.	Fresh element map, executor, text helper; quick loop.	No unrestricted computer competence. D: S1, S5; E13.
U7 Test/filter	Site, response, page → bug/policy/content signal.	Correctness oracle and blocking policy; screening.	Missed bugs/false removals. D/P: S1, S2, S5, S7, S8.
U8 Qualify leads	Form responses → fit category or score.	Business criteria and review; interactive assessment.	Bias, unclear criteria; hiring unvalidated. D/P: S1.
U9 Game actions	Structured game state/legal moves → action.	State extractor, simulator, executor; repeated choices.	Speed ≠ strategy. Doom/Melee/chess: S1, S2, S5; StarCraft hearsay: S8. E8.
U10 Simulate	Agent attributes/toy objects → behavior/bucket.	Simulation and known action space; many judgments.	Town/candy demos do not prove realism or robotics. D: S5.
U11 High stakes	Road simulation, financial/sensitive data → action/risk.	Domain validation, independent safeguards.	No deployment evidence for driving, health or military use. D/P: S1, S5, S7.
U12 Smart home	Command/device state → target and allowed action.	Device registry, permissions, executor; bounded commands.	Mock devices, not safety assurance. D/I: S8; E17.
U13 Wikiracing	Current page, target, links → next link.	Page retrieval, candidate map, stopping rule; repeated selection.	Multi-step navigation; no optimal-path proof. D: S8; E8.

What the demos add. S8 reports a 185 ms smart-home result and a 1.7 s Wikipedia run; neither specifies a repeatable timing protocol. The official smart-home design asks speculative questions together, discards irrelevant answers in code, and uses an LLM to split compound requests or converse. [S8; E17]

Two-stage selection: the vendor describes scoring then choosing when Wikipedia links exceed the 255-option cap. Retrieving the next page still creates a sequential dependency. The browser repository likewise separates Jev action selection from LLM-written text; its reported 7.073 s flight search stops at results, not booking. [E8, E13; INTERPRETATION]

Additional application candidates

7.1 What S9 adds. Most named applications extend existing families rather than establish new model abilities. The table normalizes the additional examples into reusable use cases. These are **proposals**, not validated deployments. S9 names the domains; the mappings, prerequisites and risks below are INTERPRETATION. E28 confirms that the vendor presents them as example applications.

ID / goal	Input → output	Prerequisite / possible fit	Limit and evidence
U14 Evidence support	Claim + cited passage → support judgment or rubric score.	Keep the exact evidence span; define contradiction, partial support and insufficiency. Repeated narrow checks may fit.	Support within a passage does not establish that the passage is true. P: S9/E28; extends U7.
U15 Research screening	Abstract/paper excerpt + inclusion criteria → include, exclude or review.	Expert labels and explicit criteria; reduce repeated screening effort.	Missing methods and ambiguous scope cause errors. This does not autonomously make discoveries. P: S9/E28.
U16 Recruiting screening	Application evidence + job-related rubric → criterion scores.	Validated, relevant criteria and accountable review; bounded matching may fit.	Bias and missing evidence; no evidence supports autonomous hiring decisions. P: S9/E28; extends U8.
U17 Advertising review	Campaign text + policy → label, suitability or review.	Current policy, text extraction and reviewer labels; repeated checks.	Text-only judgments omit visual context; policy errors remain possible. P: S9/E28.
U18 Insurance triage	Report + relevant policy facts → category or specialist queue.	Authorized records and policy checks; bounded case routing.	No demonstrated claim-adjudication reliability. P: S9/E28; part of U11.
U19 Financial-crime triage	Narrative + relevant records → risk indication or queue.	Domain evaluation and independent checks; prioritize investigation.	A signal is not proof of crime; both missed cases and false flags matter. P: S9/E28; part of U11.
U20 Semantic linting	Code/text + stated convention → suspected violation.	Precise guidelines and confirmed examples; repetitive semantic checks.	Cannot replace compilers, deterministic linters, tests or security review. P: S9/E28; extends U7.

Already covered. Search and retrieval map to U3; model routing to U4; LLM guardrails to U5/U7; lead generation to U8; support to U2; gaming to U9. Naming another industry does not constitute an additional benchmark. [INTERPRETATION: S9]

Input boundary. Current official documentation specifies text inputs: strings, JSON objects or arrays. Images, audio and video require preprocessing elsewhere. Browser perception, visual sorting and driving perception therefore cannot be attributed to Jev merely because the application displays them. [EXTERNAL VERIFICATION: E24/E25; INTERPRETATION]

Teaching rule. First identify the bounded judgment, then the evidence it requires, and only then the domain. A use-case label such as scientific discovery or fraud detection is too broad to define an evaluation. [TEACHING CHOICE]

Claims and evidence assessment

8 Claim ledger. Repeated launch numbers are not independent corroboration. S8 adds a user report; S9 adds a tutorial. Neither establishes a production guarantee. C17-C22 and corrected example arithmetic follow on page 10.

ID / claim	Claimant and evidence	Status	Qualification
C1 Low latency	S1/S2/S4/S7: 70-500 ms; E8.	Vendor report	Service timing, not an application deadline; load and location matter.
C2 Large speedup	S7: up to 193.6×; S1/S5 ~200×; E8/E9.	Vendor comparison	Task and comparator settings determine ratio.
C3 Lower cost	S7: up to 444.6×; S8: 20-200×; E8.	Not matched	S8 gives no cost protocol; these ranges are not interchangeable.
C4 Cheap input	S1/S2/S7; E11: $0.042 per million input tokens.	Posted rate	E11 rechecked 1 Oct 2026; provider terms can differ.
C5 Free output = free AI	S5 framing; S1 input/output slip.	Misleading	Zero output charge excludes billed inputs and system costs. [E11]
! C6 No hallucinations	S1/S5 broaden type safety; S2/S7 qualify.	Disputed meaning	Valid values can be false or harmful. E18 documents errors.
C7 Zero structure errors	S1 chart; E8 schema guarantee.	Design claim	Vendor says zero is not an empirical measurement. API failure is separate.
C8 Zero tool errors	S1 tool-call chart; E8.	Overextended	A valid tool name can still have wrong target, timing, arguments or outcome.
C9 Calibration	S2/S3/S7/S8; E7 objective.	Objective, not proof	Need held-out labels and calibration curves; confidence is not accuracy. [E14]
C10 Parallel evaluation	S1/S4/S6/S8; E1.	Documented interface	Same-state questions can run together; dependent actions require new state.
! C11 New architecture	S1/S5 assert; S2 speculates; S4 questions.	Unresolved	No disclosed layer specification, loss or ablation establishes novelty. [E16]
! C12 Self-driving	S1/S5 road simulation.	Unsupported leap	Does not establish real perception, vehicle control or safety.
C13 Production-ready	S1/S5 forecasts; S6 integration; S8 early-user story.	Not established	Needs outcomes, uptime, tail latency, failure and operational evidence.
C14 Vercel 6× / saturated eval	S8; Malte Ubl original post E19.	User report located	Indexed original confirms claim; dataset, size, metric, timing and code unavailable.
C15 Demo speed	S8: 185 ms smart home; 1.7 s Wikipedia.	Transcript only	No repeat count, timing boundary, network or matched baseline supplied.
C16 Guardrail / skill gains	S8 proposal; E20 skill-selection recipe.	Narrow vendor test	A skill-loading test is not a security audit or whole-agent success test.

Evaluation meaning. E9 compares four workflows with reference probabilities from two large models, assuming the workflow code is correct. Reference agreement is not ground truth. E10 supplies an adapter, not independent replication. E19 is an additional user report, but **saturating an eval** means reaching its useful measurement ceiling, not proving universal correctness. [EXTERNAL VERIFICATION / INTERPRETATION]

Cost arithmetic. At $0.042/M input tokens, one million requests of 1,000 billed tokens cost $42 before retrieval, hosting, retries or other models. The Doom figure of roughly $7/hour concerns the vendor's particular 10-query/s demo; it is not a flat hourly tariff. [E11; S2, S8; E8]

What the two added videos contribute

8.1 Extension assessment. Both additions are useful, but for different reasons. S8 broadens the examples and supplies an early-user anecdote. S9 provides the clearer instructional bridge from a software requirement to typed questions and application policy. Neither independently validates the launch multipliers or zero-error rhetoric. [INTERPRETATION: S8/S9]

Material	New or useful contribution	Treatment in this edition
S8: workflow overview	Smart-home commands, Wikiracing, Vercel anecdote, tool/skill routing.	Retain X3/X4, U12/U13 and C14-C16. Keep timings and user reports scoped to their sources.
S9: beginner tutorial	Rules versus semantic judgment; low Noul means likely no; distinct degrees/confidence; TypeScript walkthrough.	Add worked example and policy explanation. Correct decimal transcription and billing arithmetic.
Repetition	Three primitives, low price, parallel evaluation, games and customer triage.	Link to existing entities; do not count repetition as independent corroboration.

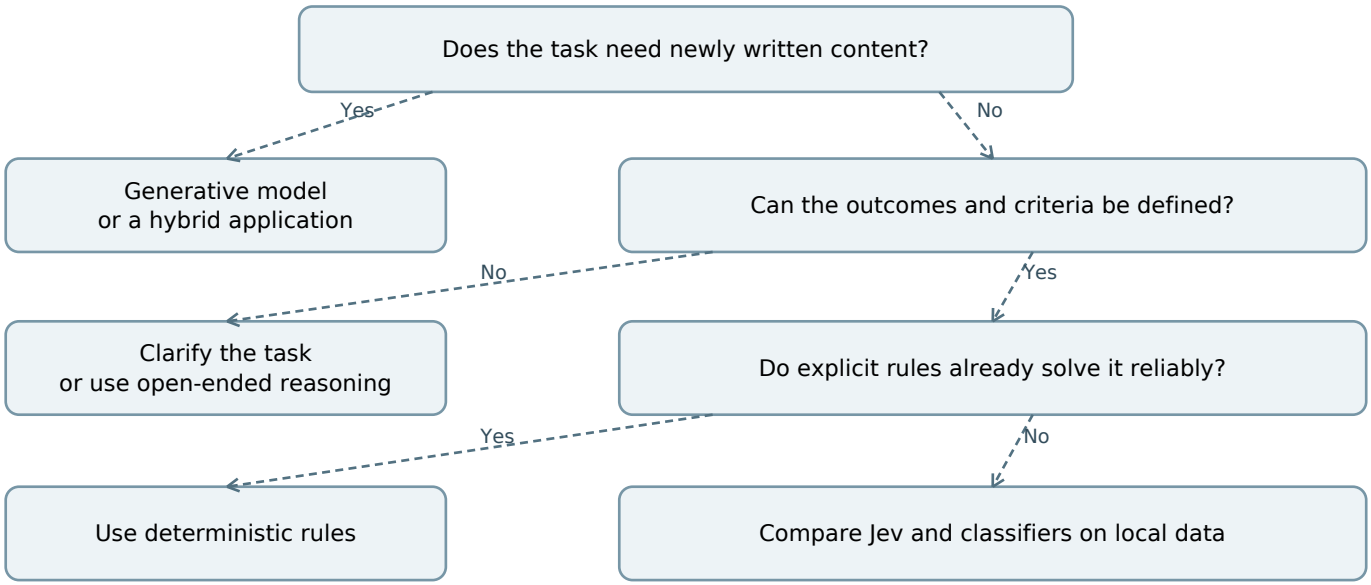
ID / claim	Source / evidence	Status and qualification
C17 $40 per billion	S9 also says $0.042 per million; E24 lists $42/B.	Numerically inconsistent. Correct conversion is $42 per billion input tokens.
C18 About 47,000 requests per $1	S9 assumes 500 input tokens per request.	Arithmetic holds approximately: 500 × $0.042 / 1,000,000 = $0.000021; $1 buys about 47,619 such requests.
C19 Displayed speed/cost comparison	S9; E8 and vendor homepage give 8.566 s / 0.114 s and $0.013880 / $0.000081.	Calculated ratios ≈75.1× faster and 171.4× cheaper. These are separate from the headline 193.6× / 444.6× workflow claims.
C20 Waitlist / free gateway window	S9 mentions early access and free use until 23 September.	Historical statements, not current onboarding guidance. The named promotional date has passed; no present free offer is inferred.
C21 All questions under one second	S9 narrated terminal example.	A demo timing, not a latency distribution, service-level guarantee or proof of unlimited throughput.
C22 Review threshold is 0.8	S9 transcribes it as “8”; matching article E26 writes 0.8.	Confirmed illustrative value. No evidence that 0.8 is optimal for another application.

Cost scope. Under C18’s assumption, 10,000 requests cost $0.21 in model input charges. Include state, questions and criteria in the billed input; more questions are not automatically free. Retrieval, preprocessing, retries and other model calls add costs. [CALCULATION / INTERPRETATION; S9; E24]

Naming adds context, not proof. S9 links Jev to William Stanley Jevons and cheaper use increasing demand; E26 repeats that explanation. Treat the economic analogy as a product thesis, not evidence of future market growth. System One borrows Kahneman’s terminology; it does not establish human-like cognition. [S9; E25/E26]

A practical mental model

9 Task suitability. TEACHING CHOICE: use a decision-oriented model when outputs can be defined in advance and the task benefits from fast, structured selection with uncertainty. Use a generative model when the result must create or explain open-ended content. Use both when a workflow contains both kinds of work. This is a design heuristic, not a quotation or a performance guarantee.



Dashed links represent instructional synthesis. “Compare” means evaluate, not deploy. A domain-specific classifier or a deterministic rule may fit better than a general question-driven model.

Accuracy and calibration answer different questions. Accuracy asks how often the selected answer is right. Calibration asks whether probability estimates match outcome frequencies. In an illustrative group assigned 0.8 probability, about 80 successes out of 100 would be consistent with calibration; the remaining errors still matter. A system can be calibrated but insufficiently accurate, or accurate while overconfident. [E14; TEACHING CHOICE]

Confidence is not a certificate. A distribution-derived confidence statistic is not automatically the empirical probability that a decision is correct. Nor does a concentrated distribution explain why the model chose an option. Thresholds should reflect observed performance and the relative costs of false positives and false negatives; uncertain cases need a meaningful fallback. [E5; INTERPRETATION]

10 Open questions and next steps

Technical questions: What are the backbone, parameter count, training mixture, RLCD reward and loss, and inference implementation? What changes across model versions? S2’s proposed prediction heads and S3’s reward discussion cannot fill these gaps. [MISSING CONTEXT: S2, S3, S7]

Evidence questions: How does Jev perform against independently labeled data, strong task-specific classifiers and concise structured-output baselines? What happens with long inputs, many questions, rare classes, adversarial content and shifted domains? Record exact prompts, versions, sample sizes, error definitions, concurrency and timing boundaries. Report p50 and p95 latency as well as quality; medians conceal slow cases. [INTERPRETATION]

Adoption questions: Is the input evidence complete? Who owns the rubric? What action requires authorization? What happens on timeout or a confident mistake? Which provider handles data, under what retention conditions? Can versions and budget be controlled? Begin with a reversible task and a labeled holdout set; compare total workflow cost and outcomes. Advanced lessons should cover calibration curves, selective prediction, decision costs and correlated errors. [TEACHING CHOICE]

A. Source register

The supplied **jev(1)(4).txt** contains nine identifiable segments. Physical line numbers below locate their boundaries. The previous available PDF already incorporated S8 but could not locate S9. This edition now registers both additions and removes that missing-source finding. IDs S1-S8 are preserved.

ID / locator / identity	Substantive contribution	Narrative, limitations, missing metadata
S1 · line 1 David Andre, self-identified	Interfaces, spreadsheets, browser/road demos, QA and lead-form build.	Business hype, Hostinger sponsorship; informal examples and unexpected outputs. Title/date/URL absent.
S2 · line 3 Unidentified speaker	Classification, routing, privacy/tool tests, variability and architectural caveats.	Subscription appeal; no controlled test set. Identity/title/date/URL unverified.
S3 · line 5 Diogo Almeida, probable	Automation thesis, RLHF critique, pretraining and calibrated decision objective.	Founder perspective and recruiting. Identity tentative; event/title/date/URL unknown.
S4 · line 6 Unidentified speaker	Optimization targets, plant sorting, narrow models and novelty critique.	JetBrains sponsor; overstated raw-text restriction. Open-model comparison and metadata unverified.
S5 · line 7 Unidentified speaker	Town/candy simulations, chess, Melee, routing, filtering and road demo.	Zapier sponsor, zero-error and free-AI rhetoric; abrupt ending, missing protocols and metadata.
S6 · lines 9-201 LangChain article match	TypeSafeClassifier, model router and risky-tool middleware.	Matched E12: Sydney Runkle/Hunter Lovell, 17 Sep 2026. Integration promotion; supplied syntax differs.
S7 · lines 203-632 Mehul Gupta attribution clue	Overview, vendor claims, hybrid workflows and specification table.	Medium promotion; article metadata not recovered. Simplified Noul schema and outdated limits.
S8 · line 634 Rob Shocks, probable match	Vercel report, smart home, Wikiracing, playground and MCP/skill routing.	Likely E22, JEV Breakdown: The First AI Model Built For Code. Exact identity/date unresolved. Hype and subscription appeal; no protocol.
S9 · line 635 Codevolution, probable match	Rules versus judgment; primitives, isolated questions, TypeScript demo, thresholds and task fit.	Launch-view framing; dated access/promotion claims and transcription errors. Matching E26 article by Vishwas Gopinath, 21 Sep 2026; E27 demo repo. Video date not established.

Boundary cues. S8 begins “so this huge new model can't write a single sentence”; S9 begins “so Jev just launched and its launch post on X”. These are locators, not invented video titles. The rest of the source material is paraphrased unless explicitly marked QUOTE.

Provenance. E26 and E27 match the tutorial’s message, filenames and three-question example. They improve traceability and confirm the 0.8 threshold; they are not independent replications. Direct retrieval of the probable S9 YouTube page was throttled, so speaker matching remains qualified. [IDENTITY UNVERIFIED in the supplied text]

Metadata policy. Missing titles, dates, affiliations and URLs remain missing. Similar launch numbers across sources are treated as a shared claim lineage, not separate experiments. References E1-E30 appear on page 16.

Canonical entities and schema

B Reusable entity register. K1-K15 (page 2) are Concepts except K10 (Metric); C1-C22 are Claims (pages 9-10); U1-U20 are Use cases (pages 7-8); S1-S9/E1-E30 are Sources (pages 12/16). Definitions, cues and status are inherited from those tables.

ID / canonical name / category	Definition, cue, aliases and provenance
A1 TypeSafe AI / Actor	Developer of Jev. Cue: company. TypeSafe, Typescafe, Typespace, Tface; E1 verified.
A2 Diogo Almeida / Actor	TypeSafe founder; InstructGPT coauthor. Cue: developer perspective. Dio Almida, Diego; E8/E15 verified; S3 probable.
M1 Jev / Model/System	Bounded decision model. Cue: evaluates questions. Jeff, eff, Jeb; S1-S8; E1 documented.
M2 TypeSafe API / Model/System	Service contract for evaluations. Cue: request/response. system_one; E6 documented.
M3 Host application / Model/System	Software enforcing policy and executing. Cue: owns actions. Surrounding software; S6-S8 interpreted.
I1 Application state / Input	Evidence supplied for evaluation. Cue: available facts. state; E1 documented.
I2 Typed questions / Input	Instructions, options and criteria. Cue: define judgment. questions; E6 documented.
O1 Typed answer / Output	Bounded value with uncertainty. Cue: result before action. answers; E6 documented.
O2 Software action / Output	Operation executed by the application. Cue: changes state. action; S6-S8 interpreted.
P1 Evaluation / Process	Assessment of supplied state. Cue: model inference. inference; E1 documented.
P2 Policy application / Process	Maps answers to action/review. Cue: what is allowed. threshold policy; E5 interpreted.
V1 Vendor workflow evaluation / Evidence	Four-workflow comparison. Cue: reference-model agreement. workflow evals; E9 verified scope.
V2 Browser repository / Evidence	Inspectable implementation/traces. Cue: inspect the harness. Jev Ultrafast; E13 verified scope.
X1 Flight search / Example	Task stopping at matching results. Cue: search, not purchase. booking in S1; E13 qualified.
X2 Road simulation / Example	Toy driving environment. Cue: simulated road. self-driving demo; S1/S5 source-stated.
L1 Semantic error / Constraint	Valid but incorrect judgment. Cue: wrong option. wrong answer; S2/S7 interpreted; E18 documented.
L2 Undisclosed internals / Constraint	Missing architecture/training details. Cue: hidden anatomy. None; S2/S7 incomplete.
R1 Defined outcomes / Prerequisite	Allowed answers and meanings. Cue: answer space first. option set/rubric; E2-E4 documented.
R2 Labeled evaluation set / Prerequisite	Held-out cases with known outcomes. Cue: correctness reference. holdout set; E14; teaching choice.
T1 Decision-loop framing / Teaching choice	Separates estimate, policy, action. Cue: code acts. None; this introduction; interpreted.

Added entity / category	Definition; cue; aliases; sources / status
K16 Speculative fan-out / Concept	Batch possibly relevant questions; code filters. Cue: ask now, use selectively. Alias: speculative questions. E17 documented; needs I1/I2/P2.
K17 Evaluation saturation / Concept	Test reaches its useful ceiling. Cue: test stops distinguishing. Alias: saturated eval. S8/E19 claim; definition interpreted; needs a test metric.
X3 Smart-home demo / Example	Bounded commands change mock devices. Cue: intent into device action. Alias: smart home. S8/E17 documented demo.
X4 Wikiracing / Example	Reach a target page via links. Cue: choose the next link. Alias: six degrees. S8/E8 vendor demo.
L3 Jev 1.13 failure modes / Constraint	Version-specific weaknesses. Cue: test rough edges. Alias: jaggedness. E18 documented; not all versions.

Closed categories: Actor; Model/System; Concept; Input; Output; Process; Claim; Evidence; Example; Use case; Constraint; Prerequisite; Metric; Source; Teaching choice. Categories separate entities from assertions and demonstrations from capabilities. Existing IDs are preserved; new records extend them.

B-C. New entities and relationships

The normalized register on page 13 is extended below. K1-K17, C1-C16, U1-U13 and their source IDs remain stable. New claims C17-C22 are on page 10; new use cases U14-U20 are on page 8. Definitions and evidential status there are inherited here.

ID / canonical name / category	Definition; cue; aliases	Sources / status / prerequisite
K18 Application threshold / Concept	Cutoff converting a model estimate into a code branch. Cue: when code changes path. Alias: review threshold.	S9/E5/E26; documented pattern. Needs K5 and explicit error costs.
K19 Question isolation / Concept	Each question evaluates the same state without reading another answer. Cue: shared evidence, separate judgment. Alias: independent evaluation.	S9/E1; documented interface. Needs I1/I2; not statistical independence.
K20 Semantic judgment / Concept	Assessment of meaning against a criterion. Cue: recognize meaning beyond keywords. Alias: smart if statement (metaphor).	S9; interpreted. Needs a question and context.
K21 Jevons paradox / Concept	Efficiency gains can raise total consumption when increased use outweighs savings. Cue: cheaper use, potentially more use. Transcript: Jean’s paradox, Jevans.	S9/E26; explanatory analogy, not an AI-demand forecast. No model capability follows.
K22 Model version / Concept	Identifier of the model release that served a request. Cue: which model answered? Alias: serving model.	S9/E24; documented. Needs API response; alias can move.
M4 TypeSafe JavaScript SDK / Model/System	Client library constructing requests and typed responses. Cue: API bridge. Alias: @typesafe-ai/sdk; garbled Typescafe AI/SDK.	S9/E23; documented. Needs runtime and credentials.
X5 Frustration tutorial / Example	One message evaluated for intent, frustration presence and degree. Cue: one input, three meanings. Alias: S9 demo.	S9/E26/E27; described demo, not reproduced here. Needs K2-K4.
V3 Tutorial repository / Evidence	Published source project for the TypeScript example. Cue: inspect the example code. Alias: gopinav/jev-demo.	E27; repository located. Demonstrates implementation, not broad quality.
R3 Representative messages / Prerequisite	Cases reflecting intended input conditions, with expected labels. Cue: test the real decision. Alias: local examples.	S9; interpreted evaluation requirement. Supports R2.

Subject	Relationship	Object	Source / status
M4	USES	M2	E23 · externally verified
U2	EXEMPLIFIED_BY	X5	S9 · source-stated
X5	SUPPORTED_BY	V3	E27 · implementation located
P1	USES	K19	E1 · externally verified
P2	USES	K18	S9/E5 · source-stated
K18	REQUIRES	R3	S9 · interpreted
U14	REQUIRES	I1	S9 · interpreted; include cited passage
C17	CONTRADICTED_BY	E24	Posted rate + arithmetic; corrected
C22	SUPPORTED_BY	E26	Written tutorial confirms 0.8

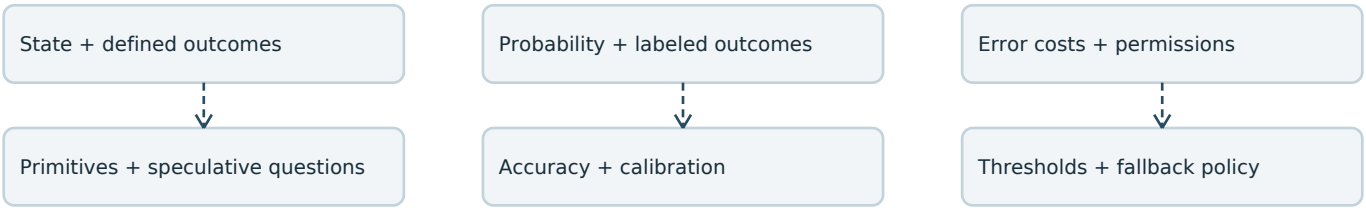
Prerequisite extension. Defined outcomes → K2-K4; probability → K18; K19 → batching design; version awareness → repeatable threshold evaluation. These are TEACHING CHOICES using PRECEDES, not claims about neural architecture.

Relationships and integrity audit

C Controlled relationships. Every endpoint resolves to a register. “Externally verified” means the published relationship was checked; it does not imply independent performance validation. No new relationship predicates are needed.

Subject	Relationship	Object	Sources / status
M1	CREATED_BY	A1	E1 · externally verified
M1	RECEIVES_INPUT	I1, I2	E1/E6 · externally verified
M1	PRODUCES	O1	E1/E6 · externally verified
P2	USES	O1	S6/E5 · source-stated
P2	PRODUCES	O2	T1 · interpreted
M1	LIMITED_BY	L1, L2, L3	S2/S7 · interpreted/incomplete; E18 · verified for 1.13
C2	SUPPORTED_BY	V1	E8/E9 · vendor scope verified
C6	QUALIFIED_BY	L1, L3	S2/S7/E18 · disputed broad meaning
C11	QUALIFIED_BY	L2	S2/S4 · incomplete/disputed
U6	EXEMPLIFIED_BY	X1	E13 · externally verified
X1	SUPPORTED_BY	V2	E13 · externally verified
K2, K3	REQUIRES	R1	E2/E3 · externally verified
K6	REQUIRES	R2	E14/T1 · interpreted
K4	DISTINCT_FROM	K12	E4/E6 · externally verified
U12	EXEMPLIFIED_BY	X3	S8/E17 · documented example
U13	EXEMPLIFIED_BY	X4	S8/E8 · documented example
X3	USES	K16	E17 · externally verified
C14	CLAIMED_BY	E19	S8/E19 · source-stated
C14	QUALIFIED_BY	K17	Teaching interpretation
C15	EXEMPLIFIED_BY	X3, X4	S8 · source-stated timings, incomplete protocol
C16	SUPPORTED_BY	E20	Vendor skill-selection evidence only; security claim incomplete

D Prerequisite map. Dashed arrows mean PRECEDES in a teaching sequence, not neural architecture. Solid arrows elsewhere mark documented relationships; “!” flags disputed or incomplete evidence.



E Quality-control log and extension assessment.

Normalization: existing Jev/TypeSafe/RLHF variants retained. S8’s “new” → Noul only in primitive context; “Forcel/Verscell” → Vercel and “flashlight” → Gemini 2.5 Flash Lite in the matched E19 claim. “System One” is the model category, not the company name. “ChatGPT co-founder” is replaced with the evidenced InstructGPT coauthor description. [E1, E8, E15, E19]

New information: S8 adds X3/X4, C14/C15 and skill-routing context. E17 supplies the missing LLM fallback and speculative-question pattern. E20 documents a skill-suggestion implementation, while E21 supports bounded argument selection. These qualify the earlier blanket tool-selection framing; they do not establish arbitrary text generation.

Gaps and disagreements: S9 is now present; the earlier missing-transcript finding is closed. S8’s cost range, StarCraft, email throughput and Open Code attribution still lack matched tests. S9’s decimal errors, pricing and expired promotion are qualified on page 10; metadata remains partial.

Preserved audit: Noul ≠ null; probability ≠ confidence ≠ calibration ≠ accuracy. S4’s raw-text restriction, S7’s Noul schema/context limit and S1’s pricing slip remain corrected. Novelty and zero-hallucination rhetoric conflict with narrower interface evidence. E18 supplies concrete counterexamples to universal reliability; no measured failure rate follows. Independent calibration, matched benchmarks, training details and safety-critical deployment remain unresolved.

External references and access record

Each title below is a clickable link. External verification establishes what a document says; vendor documentation and demonstrations do not independently establish performance. S1-S9 refer to the supplied source collection, not the websites.

E1 · TypeSafe: Introduction

E2 · TypeSafe: Choice

E3 · TypeSafe: Score

E4 · TypeSafe: Noul

E5 · TypeSafe: Confidence

E6 · TypeSafe: API reference

E7 · TypeSafe: AI primer

E8 · TypeSafe: Introducing System One Models & Jev

E9 · TypeSafe: Workflow evaluations

E10 · TypeSafe: System One Adapter repository

E11 · Vercel: Classify, route and score with Jev

E12 · LangChain: Building a harness with Jev

E13 · Browser Use: Jev Ultrafast repository

E14 · Guo et al. (2017): On Calibration of Modern Neural Networks

E15 · Ouyang et al. (2022): Training language models to follow instructions

E16 · Devlin et al. (2018): BERT

E17 · TypeSafe: Smart home assistant demo

E18 · TypeSafe: Jev 1.13 jaggedness

E19 · Malte Ubl: Early-user classifier evaluation report

E20 · TypeSafe: Skill suggestion cookbook

E21 · TypeSafe: Function calling cookbook

E22 · JEV Breakdown: The First AI Model Built For Code (probable S8)

E23 · TypeSafe: JavaScript SDK

E24 · TypeSafe: Models, limits, aliases and pricing

E25 · TypeSafe: System One definition and input boundary

E26 · Vishwas Gopinath / Builder.io: What is Jev and how to use it

E27 · gopinav: Jev TypeScript demo repository

E28 · TypeSafe: Example use cases

E29 · TypeSafe: Quick start

E30 · TypeSafe JavaScript API: noul()

Access record. This update checked E1-E5, E7-E9, E11-E12, E17-E18, E22-E30 on 1 October 2026. E6, E10 and E13-E16 retain the previous edition’s 20 September record; E19-E21 retain its 1 October record. E19 was indexed but its direct retrieval was blocked in that edition; its methodology remains unavailable. E22 returned title-level metadata only.

Revision scope. The available prior ten-page edition is the continuity baseline. The extension adds S9, worked implementation guidance, additional application candidates, C17-C22, normalized entities and corrections. Broad architectural novelty, calibrated correctness on the reader’s task, and safe deployment of consequential actions remain open questions.